

Step 3 — Test-Coverage Reviewer + Supervisor — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Nudges, not answers — try genuinely hard before Hint 2. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — What you're actually reusing from Project 4](#)
- [Hint 2 — A nudge on the compiling step, not the code](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're actually reusing from Project 4

Basic Version

You've already built exactly this shape once — Project 4 Step 5's Supervisor, routing to specialists via `Command`, with shared state. Go reread that step's own solution before designing anything new here. The question worth sitting with: what's actually *different* about CodeGuard's Supervisor, versus what you could copy over almost unchanged?

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

ContentForge's Supervisor always routed to every specialist, in a fixed order, every time — Research, then Analysis, then Writer/Reviewer. CodeGuard's problems table says something different should happen here: "Route based on which files actually changed... a `.md` -only diff doesn't need a security check." That's a real design difference, not a copy-paste job. Think about what information the Supervisor needs about a diff *before* it can make that routing decision, and where that information would have to come from.

For the Test-Coverage reviewer itself: think about what signal actually tells you "this changed code has a test," given only a diff — you don't have the whole repository to search through, just the diff text itself. What's a heuristic, even an imperfect one, that's genuinely usable from that alone?

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The problems table's last row calls the compiling step "a real task on its own — remove overlapping findings, sort by how serious they are, and write one clear comment." Before building that, go look at what 3 real reviewers' outputs actually look like side by side — run Security, Style, and Test-Coverage on the same made-up diff and print all 3 results next to each other. What do you notice? Is anything actually duplicated, or near-duplicated, across reviewers? What would "duplicated" even mean when 2 reviewers describe the same underlying problem in different words?

Difference between Basic, Intermediate, and Advanced: Basic tells you exactly where the closest working precedent is. Intermediate asks you to identify what genuinely differs about this Supervisor's job versus ContentForge's, and to design your own coverage heuristic from a real constraint (diff-only visibility). Advanced asks you to look at real output from your own 3 reviewers before designing the compiling logic, instead of guessing what "overlapping findings" means in the abstract.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — A nudge on the compiling step, not the code

Basic Version

The compiling step is its own function, with its own real job — it's not something that happens automatically just because 3 reviewers ran. Design it deliberately: what goes in (3 reviewers' findings), what comes out (one clean comment), and what the sorting/dedup rule actually is.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Reuse Project 4's `completed_steps` idea for tracking which reviewers actually ran (some diffs skip some reviewers now, on purpose) — but think about what "routing based on which files changed" needs as an *input* to the Supervisor that ContentForge's Supervisor never needed: something that looks at file extensions or paths in the diff and decides which reviewers apply, before any reviewer runs at all.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The README mentions an optional stretch: "the Supervisor can ask a reviewer to double-check an unclear finding before finishing." Before building that, think hard about what "unclear" would even mean as something your code could check — a low-confidence field the reviewer sets itself? A finding with unusually short reasoning? However you define it, remember Project 4 Step 2's exact lesson about the Writer ↔ Reviewer loop: any loop like this needs a hard limit and a way to stop making the same request twice with no new outcome, or it's the same infinite-loop risk in a new shape.

Difference between Basic, Intermediate, and Advanced: Basic tells you to treat compiling as a real, designed function. Intermediate connects it to a genuinely new requirement (file-based routing) that Project 4 never needed. Advanced asks you to design the optional double-check loop with the same bounded-loop discipline Project 4 Step 2 already taught you, rather than treating it as a free extra with no failure mode of its own.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)